

Audion Browsers Portable

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

Version 1.0.0 · 2026-08-24 · 81.6 MB

- [Direct download](#) — unmetered, no rate limits
- [Project page](#) — every version and how to install

SHA-256: d3aa5a7ed0fcbb3a6f263f5d2148a9825578e93030c79c673f4aa6c0499611ff

One engine for the whole Chromium stack: it downloads, unpacks, assembles portable browsers and then keeps them updated. Nothing is installed into Windows — installers are unpacked rather than run, and the profile lives in the build.

Who is on the list, and why

A browser belongs here under one rule: **the vendor ships no portable build that updates itself.**

Browser	What this program adds
Google Chrome	There is no portable build at all.
Yandex Browser	No portable build, and it already trusts the Russian Trusted CA.
Brave	Only third-party portable builds exist, and none of them updates.
Chromium-Gost	Speaks GOST TLS, which government portals require; a zip exists, an updater does not.
Ungoogled Chromium	A zip exists, an updater does not — that is the point of the project.

Absent on purpose:

- **Vivaldi** — Install Standalone is official and its automatic updates work in it;
- **Cent Browser** — official portable SFX with a built-in updater;
- **Opera** — no separate portable file, but Autoupdate packages sit beside every Setup, so it updates itself;
- **Thorium** — tried and dropped: the main repository publishes releases with no files, the fork ships only a 32-bit build, and the Chrome Web Store does not work in it (a plain uBlock Origin install fails).

How it works

Two shapes cover every entry:

```
installer.exe -> payload .7z -> browser folder   7
archive .zip   -> browser folder                 └-> <build>\App\
```

Chrome++ provides the portability: its `version.dll` goes beside the executable. The profile lands in `Data` beside `App`, the cache in `Cache`.

The wrapper is a choice: Chrome++, the proxy library ([neyrostalker/proksi-biblioteka](#) on GitFlic, pulled off the public pages without a token). The proxy library blocks registry writes instead of wiping the branch on exit and draws no complaint from Microsoft; it ships x86 and x64 only. The three engines and the VirusTotal check are covered in `docs/CHROME_PLUS_AND_DEFENDER.md`.

Chrome++ is a long-standing, respected open-source project; antivirus sometimes mistakes its `version.dll` for a threat. Why that is a false positive and how the program works around it during a build — see [CHROME_PLUS_AND_DEFENDER.md](#).

```
<Browser> Portable\
  App\                browser, version.dll, chrome++.ini
  Data\               profile
  Cache\              cache
  Certificates\       the certificates and two wrappers (when enabled)
  <Browser> Portable.cmd launcher
  Portable-Build.json which versions are inside
```

Architecture is not a detail

Chrome++ ships one `version.dll` per architecture. A 32-bit browser **does not load** a 64-bit one: Windows quietly falls back to the system copy, the browser starts perfectly, and the profile goes to `%LOCALAPPDATA%`. There is no error at all — the build simply stops being portable.

So the architecture is read from the browser's own PE header and the matching wrapper is used. A hand-forced mismatch aborts the build with an explanation instead of shipping it.

The interface

The root window is a switcher of four tabs: `Install`, `Update`, `Certificate`, `Service`. A command with parameters unfolds on the tab itself — its own run button, named after the action, and its own fields; there is no child window with an identical `Run` any more. Service operations (the folder cleanups) sit in a strip above the tabs: they belong to the program rather than to a tab.

Choices are switch buttons: the chosen one washed with translucent blue, the rest outlined, and browsers outlined each in its own colour. Every checkbox is a card in the block's grid, and the block carries a coloured marker. Captions are short; the explanation lives in the tooltip.

Updating

`Check` compares published versions with the builds on disk (the folder in Source first, then `output\Portable`). Nothing is downloaded: the version comes from a release tag, from the Chrome version history API, or from the redirect Yandex answers with.

`Update` replaces `App` **in place**, in the same folder the build was taken from, and keeps `Data` and `Cache`. The Target folder is not used for updates — it is for new builds. A build whose browser and Chrome++ are both current is skipped without downloading.

The replacement goes through a rename: the old `App` steps aside, the new one takes its place, and only then is the old one deleted. When the browser is running and the folder cannot be renamed, the operation says so and leaves the build alone.

Vendors number releases differently, and that is accounted for: a release tagged `1.18.2` ships a file reporting `1.18.2.0`, while Brave tags `1.93.134` and installs a `brave.exe` version `151.1.93.134`, where `151` is the Chromium major.

The certificate block

An option, and a reversible one. Building places both certificates and two wrappers into the folder. Trust is added by a separate command:

```
certutil -addstore -user -f Root russian_trusted_root_ca.crt
certutil -addstore -user -f CA russian_trusted_sub_ca.crt
```

The current user's store: no administrator rights, no effect on other accounts. Removal goes by thumbprint, so it takes back exactly what it added. Both operations re-read the store afterwards rather than trusting an exit code.

Yandex Browser needs none of this — it trusts that CA out of the box.

What it leaves in the system

Every browser keeps counters in its own `HKCU` branch. The build can wipe it on exit, but does not by default: that branch is shared with an installed copy of the same browser.

Requirements

- Windows, the portable Python in `runtime\` (ships with the project).
- `tools\7zip\bin\7za.exe` — installed by the `7-Zip` command on the `Service` tab.
- 120–240 MB of download per browser, 400–500 MB per build on disk.

Running

```
launcher_gui.cmd
```